CSS4003 — DevOps 2

Lecture 9: Cloud Storage & Docker Fundamentals

Nauruzbayeva Farikha · Lesson 9



Recap — Lesson 8 (AWS Networking)

- What makes a subnet "public"? (its route table sends `0.0.0.0/0` to an internet gateway — not its name, not its CIDR)
- How many addresses does a `/24` subnet have? (256, of which AWS reserves 5 — so 251 usable)
- Why does SG-app allow SSH from SG-web instead of an IP? (it follows the web servers wherever they move; an IP rule breaks when an instance is replaced)
- Security group vs NACL: which one is stateful? (the security group — return traffic is allowed automatically)

Today’s agenda

- ☐ Part 1 — cloud storage in 15 minutes: EBS, S3, EFS, RDS, DynamoDB
- ☐ Scripting S3: a backup script, boto3, and where credentials come from
- ☐ Part 2 — why containers, and how they differ from VMs
- ☐ Images, containers, registries and the everyday `docker` commands
- ☐ Writing a Dockerfile, and making the layer cache work for you
- ☐ Volumes, networks and Docker Compose
- ☐ Storage meets containers: a container that backs up to S3
- ☐ Live demo → straight into today’s practice

The scenario

Your teammate’s Flask app "works on my machine". On the server it crashes: a different Python version, a missing library. When it finally runs, the notes users save live on the server’s disk — until someone redeploys and they’re gone.

Today: package the app so it runs the same everywhere — and keep its data somewhere that survives.

Part 1

Cloud storage in 15 minutes

Block, file, object

	Block	File	Object
Looks like	A raw disk	A shared folder	A key → a blob of bytes
You use it via	A filesystem you format	NFS mount	HTTP API (`GET` , `PUT`)
Attached to	One instance	Many instances	Nothing — reach it from anywhere
AWS service	**EBS**	**EFS**	**S3**
Good for	OS disk, databases	Shared app files	Uploads, backups, logs, static sites

The scenario's mistake: user data lived only on the server's own disk, with no copy anywhere else.

EBS vs S3 vs EFS

	EBS	S3	EFS
Scope	One AZ, one instance at a time	Region, reached over HTTPS	Region, many instances
Survives instance stop	Yes	—	—
Backups	Snapshots (incremental)	Versioning, replication	AWS Backup
Typical DevOps use	Root disk, DB data files	Artifacts, backups, static sites	Shared uploads folder

- EBS volume types differ in speed and price: **gp3** is the general default.
- **Instance store** is a fast local disk that is **lost on stop** — scratch data only.

S3 essentials: versioning and lifecycle

- **Versioning on** → an overwrite keeps the old copy; `rm` adds a **delete marker** instead of erasing. Remove the marker → the file is back.
- Every version is billed, so pair it with a **lifecycle rule** that expires old versions and moves old data to cheaper classes.

```
aws s3api put-bucket-versioning --bucket lab09-123 \
  --versioning-configuration Status=Enabled
aws s3api put-bucket-lifecycle-configuration --bucket lab09-123 \
  --lifecycle-configuration file://lifecycle.json   # rules live in Git
```

S3 essentials: keep it private, share briefly

- New buckets are **private**. **Block Public Access** (all four switches on) stops any policy or ACL from making them public by mistake.
- To share **one** file, hand out a **presigned URL**: a link with a signature and an expiry, using **your** permissions.

```
aws s3api put-public-access-block --bucket lab09-123 \
  --public-access-block-configuration \
  BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=true
aws s3 presign s3://lab09-123/report.pdf --expires-in 300   # valid 5 minutes
```

S3 from the CLI — what scripts and CI/CD use

```
aws s3 ls s3://lab09-123/                              # list a prefix
aws s3 cp build.zip s3://lab09-123/releases/v1.zip     # upload one file
aws s3 cp ./data s3://lab09-123/backup/ --recursive    # upload a folder
aws s3 sync ./dist s3://lab09-123/site/ --delete       # mirror (careful!)
aws s3api get-bucket-versioning --bucket lab09-123     # low-level API
```

- `aws s3` — file-like commands: `ls`, `cp`, `sync`, `rm`.
- `aws s3api` — one command per API call: versioning, lifecycle, policies.
- `sync --delete` removes what isn't in your folder: run `--dryrun` first.

Databases: Amazon RDS

	DB on EC2	Amazon RDS
OS and DB patching	You	AWS
Backups	You script them	Built in, with point-in-time restore
Standby in another AZ	You build replication	One setting: **Multi-AZ**
Your job	Everything	Schema, data, who can connect, cost

- Runs in **private subnets** of your VPC; you connect to an **endpoint** (DNS name).
- **Multi-AZ** = availability (same endpoint). **Read replica** = more reads (own endpoint).

Databases: DynamoDB

- Fully managed **key-value / document** database: no servers, no patching.
- Every item has a **partition key** (e.g. `id`); other attributes are free-form.
- **On-demand** capacity: pay per request — good for spiky traffic.

```
aws dynamodb put-item --table-name students \
  --item '{"id":{"S":"s001"},"name":{"S":"Aigerim"}}'
aws dynamodb get-item --table-name students --key '{"id":{"S":"s001"}}'
```

Which one for which data?

Data	Service	Why
OS disk of an instance	EBS (gp3)	Block disk, survives stop/start
Uploads, backups, logs, artifacts	S3 (+ lifecycle)	Durable, cheap, reachable from anywhere
Files shared by several servers	EFS	One file system, many instances
Relational app data	RDS (Multi-AZ in production)	SQL, joins, managed backups
Simple key lookups at scale	DynamoDB	Serverless, pay per request

`backup.sh` (1/2): fail fast, name things

```
#!/usr/bin/env bash
# Back up a folder to S3 under backups/<today>/
# usage: ./backup.sh <folder> <bucket>
set -euo pipefail

SRC=${1:?usage: backup.sh <folder> <bucket>}
BUCKET=${2:?usage: backup.sh <folder> <bucket>}
STAMP=$(date +%F-%H%M%S)                  # 2026-11-03-101500
NAME="$(basename "$SRC")-$STAMP.tar.gz"
KEY="backups/$(date +%F)/$NAME"           # one "folder" per day
```

`-e` stops on the first failed command, `-u` on an unset variable, `-o pipefail` when any part of a pipe fails. `${1:?msg}` exits with a usage message if the argument is missing.

`backup.sh` (2/2): pack, upload, show

```
tar -czf "/tmp/$NAME" -C "$(dirname "$SRC")" "$(basename "$SRC")"
aws s3 cp "/tmp/$NAME" "s3://$BUCKET/$KEY"
echo "Backed up $SRC to s3://$BUCKET/$KEY"
aws s3 ls "s3://$BUCKET/backups/" --recursive
```

```
$ ./backup.sh myapp lab09-123
upload: /tmp/myapp-2026-11-03-101500.tar.gz to s3://lab09-123/backups/2026-11-03/...
Backed up myapp to s3://lab09-123/backups/2026-11-03/myapp-2026-11-03-101500.tar.gz
```

Thanks to `set -e`, a failed upload stops the script — it never prints "Backed up" for a backup that doesn't exist. Later today the same idea runs inside a container.

Python + boto3: S3

```
import boto3
s3 = boto3.client("s3")

s3.upload_file("notes.txt", "lab09-123", "docs/notes.txt")
s3.download_file("lab09-123", "docs/notes.txt", "copy.txt")
url = s3.generate_presigned_url("get_object",
        Params={"Bucket": "lab09-123", "Key": "docs/notes.txt"},
        ExpiresIn=300)                # seconds
```

Upload, download, presigned URL — the CLI commands, but inside a program that can loop, check results and decide.

Credentials: never in the code

- The AWS CLI and boto3 look for credentials in a fixed order: **environment variables** → `~/.aws/credentials` → **instance role** on EC2.
- So the same script runs on your laptop, in Learner Lab and on EC2 — with **zero keys in the file**. Region: `AWS_DEFAULT_REGION=us-east-1`.
- `AWS_ENDPOINT_URL=http://localhost:5050` sends every call to **moto**, a local S3, instead of AWS. Remember this one — containers use it today.

`aws_access_key_id="AKIA..."` in a file that reaches GitHub is a leaked key within minutes. Bots scan public repos for them.

Quick check

Versioning is on. You run `aws s3 rm s3://bucket/notes.txt`. Is the file gone?

No. S3 adds a **delete marker** as the newest version. The older versions are still there — delete the marker and the file is back.

Part 2

Docker fundamentals

"It works on my machine"

- An app needs more than its code: a **language runtime**, **libraries** with exact versions, OS packages, config.
- Every laptop and server has a slightly different mix → "works here, crashes there".
- A **container** ships the app **together with** everything it needs, as one package that runs the same on any machine with Docker.
- For DevOps this means: **build once, run anywhere** — laptop, CI, EC2, Kubernetes.

Containers vs virtual machines

	Virtual machine	Container
Contains	A whole OS + kernel + app	App + its libraries, shares the host kernel
Size	Gigabytes	Often tens of megabytes
Starts in	Minutes	Seconds or less
Isolation	Strong (hypervisor)	Process-level (namespaces, cgroups)
On AWS	EC2 instance	Runs **on** an EC2 instance (or ECS, EKS)

Containers don't replace VMs: in the cloud you usually run containers **on** VMs. The container makes the app portable; the VM provides the machine.

Image vs container

- An **image** is a read-only template: a filesystem snapshot + metadata (which command to start). Built once, stored, shared.
- A **container** is a **running instance** of an image, with its own thin writable layer on top.
- One image → many containers. Delete a container and its writable layer is gone; the image stays.
- Analogy: image = class, container = object. Or: image = `.iso`, container = the running machine.

Registries and tags

- A **registry** stores images: **Docker Hub** (public default), GitHub Container Registry, AWS **ECR**.
- Image names: `[registry/]repository:tag` — `nginx:1.31-alpine`, `python:3.12-slim`, `ghcr.io/org/app:2.0`.
- No tag means `:latest` — which changes under your feet. **Pin a version** in anything you deploy.
- `alpine` / `slim` variants are smaller base images: less to download, fewer packages to patch.

Your first container

```
docker run -d --name web -p 8080:80 nginx:1.31-alpine
docker ps                           # running containers
curl -s localhost:8080 | grep title # <title>Welcome to nginx!</title>
docker ps -a                        # also the stopped ones
```

- `run` = pull the image if missing, create a container, start it.
- `-d` — detached, in the background. `--name` — so you don't copy IDs.
- `-p 8080:80` — **host port 8080 → container port 80**. Without `-p`, nothing outside the container can reach it.

Inspect, get inside, stop

```
docker logs web                # stdout/stderr of the main process
docker logs -f web             # follow, like tail -f
docker exec -it web sh         # a shell INSIDE the running container
docker stop web                # graceful stop (SIGTERM, then SIGKILL)
docker rm web                  # delete it (or: docker rm -f web)
```

- Containers should log to **stdout**, not to files — `docker logs` (and later CloudWatch) collects it.
- `exec -it` is for debugging. Never "fix" a container by hand: rebuild the image.

Working with images

```
docker pull python:3.12-slim     # download from the registry
docker images                    # what's stored locally, with sizes
docker history python:3.12-slim  # the layers an image is made of
docker rmi lab09-app:slow        # delete a local image
```

Images are cached locally after the first pull — the second `docker run` of the same tag starts instantly.

Quick check

You ran `docker run -d nginx:1.31-alpine` and `curl localhost:80` fails. The container is running. Why?

No `-p`: port 80 exists only inside the container's network. Publish it with `-p 8080:80` and curl `localhost:8080`.

Building images

The Dockerfile

Dockerfile instructions you need today

Instruction	What it does
`FROM python:3.12-slim`	Base image to start from
`WORKDIR /app`	Working directory for the next steps (created if missing)
`COPY src dest`	Copy files from the build folder into the image
`RUN pip install …`	Run a command **at build time**, save the result as a layer
`ENV KEY=value`	Environment variable for build and run
`EXPOSE 8000`	Documents the port — does **not** publish it
`CMD ["python", "app.py"]`	Default command when a container **starts**

The app we'll package

```
import os, socket
from flask import Flask

app = Flask(__name__)
DATA = os.environ.get("DATA_DIR", "/data")

@app.get("/")
def index():
    os.makedirs(DATA, exist_ok=True)
    return {"host": socket.gethostname(), "files": sorted(os.listdir(DATA))}

@app.post("/notes/<name>")
def add_note(name):
    os.makedirs(DATA, exist_ok=True)
    with open(os.path.join(DATA, f"{name}.txt"), "w") as f:
        f.write(f"note {name}\n")
    return {"saved": f"{name}.txt"}, 201

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=8000)
```

The Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
ENV DATA_DIR=/data
EXPOSE 8000
CMD ["python", "app.py"]
```

- `requirements.txt` contains one line: `flask==3.1.3` — pinned.
- The app binds `0.0.0.0`: inside a container, `127.0.0.1` would only accept connections from the container itself.

Build and run

```
docker build -t lab09-app:1.0 .
docker run -d --name app -p 8000:8000 lab09-app:1.0
curl -s localhost:8000/                       # {"files":[],"host":"32d95490db47"}
curl -s -X POST localhost:8000/notes/first    # {"saved":"first.txt"}
curl -s localhost:8000/                       # {"files":["first.txt"],"host":"32d95490db47"}
```

- `-t name:tag` names the image; `.` is the **build context** sent to Docker.
- `host` is the **container's** hostname — its short ID, not your laptop.

Layers and the build cache

- Each `FROM`, `COPY` and `RUN` produces a **layer**. Layers are cached.
- On a rebuild Docker reuses every layer **until the first step whose input changed** — that step and everything after it run again.
- `COPY` changes when the copied files change; `RUN` when the step above it changed.
- So: put what changes **rarely** (dependencies) **first**, what changes **often** (your code) **last**.

```
#8 [4/5] RUN pip install --no-cache-dir -r requirements.txt
#8 CACHED
#9 [5/5] COPY app.py .
```

Order matters

Slow — any code change reinstalls everything

```
FROM python:3.12-slim
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
CMD ["python", "app.py"]
```

Fast — dependencies stay cached

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY app.py .
CMD ["python", "app.py"]
```

In CI this is the difference between a 10-second and a 3-minute build — on every push.

`.dockerignore`

```
.venv/
__pycache__/
.git/
.env
*.tar.gz
```

- Everything in the build folder is sent to Docker as the **build context**.
- `.dockerignore` keeps out junk (faster builds, smaller images) and **secrets** — a `.env` copied by `COPY . .` would end up inside the image.

`CMD` vs `ENTRYPOINT`

- `CMD` — the default command. Replaced by anything you type after the image name: `docker run lab09-app:1.0 python -V` runs `python -V` instead.
- `ENTRYPOINT` — the fixed program; `CMD` / your arguments become its **arguments**. Used by tool images: `amazon/aws-cli` has entrypoint `aws`, so `docker run amazon/aws-cli:2.39.3 s3 ls` runs `aws s3 ls`.
- To run something else in such an image, override it: `--entrypoint sh`.
- Use the JSON form `["python", "app.py"]`: the app gets signals (`docker stop`) directly, without a shell in between.

Tag and push (concept)

```
docker tag lab09-app:1.0 myuser/lab09-app:1.0   # name it for Docker Hub
docker login                                     # once per machine
docker push myuser/lab09-app:1.0                 # upload the layers
docker pull myuser/lab09-app:1.0                 # on any other machine
```

- The server never builds anything: it **pulls** the exact image you tested.
- Next week: the same with **AWS ECR**, and running the image on EC2.

Quick check

You changed one line in `app.py` and rebuilt. With the "fast" Dockerfile, which steps run again?

Only `COPY app.py .` and the steps after it. `FROM`, `COPY requirements.txt` and `pip install` come from the cache, because their inputs didn't change.

Data, networks, Compose

Containers that keep data and talk to each other

Containers are disposable — data isn't

- A container's writable layer is deleted with the container. Redeploy = new container = **empty** again (today's scenario).
- A **volume** is storage managed by Docker, mounted into the container at a path. It outlives any container.

```
docker volume create lab09-data
docker run -d --name app -p 8000:8000 -v lab09-data:/data lab09-app:1.0
docker rm -f app        # container gone…
docker run -d --name app -p 8000:8000 -v lab09-data:/data lab09-app:1.0   # …files still there
```

Named volume vs bind mount

	Named volume	Bind mount
Syntax	`-v lab09-data:/data`	`-v "$PWD/site":/usr/share/nginx/html`
Lives in	Docker's storage area	A folder **you** choose on the host
Created by	Docker (automatically if missing)	You
Good for	Databases, app data	Code and config during development

A typo in a volume name doesn't fail — Docker silently creates a new, empty volume. Check with `docker volume ls`.

Networks: containers find each other by name

- Containers on the same **user-defined network** reach each other by **container (or service) name** — Docker runs a DNS for them.
- No published ports needed between them: `-p` is only for traffic from **outside**.
- Never hardcode container IPs: they change on every restart.

```
docker network create labnet
docker run -d --name s3 --network labnet motoserver/moto:5.2.5
docker run --rm --network labnet python:3.12-slim getent hosts s3   # <its IP>  s3
```

Docker Compose: the whole stack in one file

- Real apps are several containers: the app, a database, a cache, a local S3…
- **Compose** describes them in `compose.yaml`: images or builds, ports, volumes, env — and puts them on **one network** automatically.
- One command starts everything; the file lives in Git next to the code.

`compose.yaml` : app + local S3

```yaml
services:
  app:
    build: .
    image: lab09-app:1.0
    ports: ["8000:8000"]
    volumes:
      - appdata:/data

  s3:
    image: motoserver/moto:5.2.5
    ports:
      - "5050:5000"   # host 5050 (5000 is AirPlay on macOS)

volumes:
  appdata:
```

Inside the stack, the app reaches the S3 service at `http://s3:5000` — service name + container port.

Compose commands

```
docker compose up -d --build        # build if needed, start everything
docker compose ps                   # status of the services
docker compose logs -f app          # logs of one service
docker compose exec app sh          # shell in a running service
docker compose run --rm backup      # start a one-off job, delete it after
docker compose down                 # stop + remove containers and network
docker compose down -v              # …and the volumes (= your data!)
```

Storage meets containers

```
backup:
  image: amazon/aws-cli:2.39.3
  profiles: ["tools"]          # not started by "up", only by "run"
  env_file: .env               # keys + AWS_ENDPOINT_URL — not in the image
  volumes:
    - appdata:/data:ro         # the app's data, read-only
  entrypoint: ["/bin/sh", "-c"]
  command:
    - |
      aws s3 mb "s3://$$BUCKET" 2>/dev/null || true
      aws s3 cp /data "s3://$$BUCKET/backups/$$(date +%F)/" --recursive
```

With `AWS_ENDPOINT_URL=http://s3:5000` in `.env` it backs up to the local moto container; remove that line and put Learner Lab keys there — the same container backs up to real S3.

Secrets: never baked into an image

- Anything you `COPY` or `ENV` into an image is readable by everyone who can pull it: `docker history`, or just `docker run … env`.
- Pass secrets **at run time**: `-e`, `--env-file .env`, Compose `env_file:` — and keep `.env` in `.gitignore` **and** `.dockerignore`.
- In AWS, containers get credentials from an **IAM role** (instance or task role) — no keys at all. Same idea as last week's credentials chain.

A pushed image with an access key inside is a public leak — deleting the tag doesn't help, the image may already be pulled. Rotate the key.

Cleaning up

```
docker ps -a                  # stopped containers pile up
docker system df              # how much space images, containers, volumes use
docker image prune            # remove dangling (untagged) images
docker system prune           # stopped containers, unused networks, dangling images
```

`docker system prune -a --volumes` also deletes **all** unused images and **volumes** — your databases' data included. Read the prompt before typing `y`.

When it doesn’t work

Symptom	Likely cause
`port is already allocated`	An old container still uses the port: `docker ps -a`
`curl` gets no reply	App binds `127.0.0.1` inside the container, or it exited: `docker logs`
Data gone after redeploy	No volume, or a typo created a new empty one
`pip install` on every build	Code copied before requirements
Container can’t reach `localhost:5050`	Inside a container, use the service name: `http://s3:5000`

Quick check

You run `docker compose down -v` at the end of the day. Tomorrow the app starts with no notes. Bug?

No — `-v` deleted the `appdata` volume. Plain `docker compose down` keeps volumes. (And that's why the backup to S3 exists.)

Try it yourself

Part A — Docker basics (everyone)

1. `nginx` with `-p` , `logs` , `exec`
2. Write the Dockerfile for the Flask app, build, run
3. Edit `app.py` , rebuild: see `CACHED`
4. Named volume: data survives `docker rm`

Part B — Compose + S3

5. `compose.yaml` with `app` + `s3`
6. One-off `backup` container → S3

Track B — local S3 (moto container):

```
AWS_ENDPOINT_URL=http://s3:5000
docker compose run --rm backup
```

Track A — real S3 in **Learner Lab** (`us-east-1`): same compose file, Learner Lab keys in `.env` , no endpoint line. Delete the bucket and **End Lab** after.

By the end of this lesson, you should be able to

- ☐ Choose between EBS, S3, EFS, RDS and DynamoDB for a given kind of data
- ☐ Script S3 with the CLI and boto3 without putting keys in the code
- ☐ Explain images vs containers, and containers vs VMs
- ☐ Run, inspect, debug and remove containers with the `docker` CLI
- ☐ Write a Dockerfile that uses the layer cache well
- ☐ Keep data in volumes and connect services with Compose
- ☐ Back up a container's data to S3 with credentials passed at run time

Before next lecture

- ☐ Finish the practice and clean up: `docker compose down -v`, remove the lab images and volume
- ☐ Track A: delete your bucket and **End Lab**
- ☐ Create a free **Docker Hub** account if you don’t have one

Next week we take today's image to AWS: push it to a registry and run it on EC2.

Next lecture

Docker + AWS — pushing images to a registry and running containers on EC2.